



INTERNAL

Collaboration Guide

Quenyx vOPS HUB — v1.0.0 RC1 · Document v1.0

Document Version	1.0
Software Version	v1.0.0 RC1
Applies To	Quenyx vOPS HUB v1.0.0 RC1 — Sprint 24
Classification	Internal
Owner	Platform Engineering
Status	Released
Document Type	Platform capability guide

Table of Contents

- | [What the Collaboration Platform is](#)
- | [How it works](#)
- | [Reuse in the UI](#)
- | [API \(UUID-only, workspace required\)](#)
- | [Guarantees](#)

Collaboration Guide

What the Collaboration Platform is

A **shared, reusable platform capability** (Sprint 24) that adds comments, mentions, assignments, watchers, shared investigations, and task ownership to **any entity** in the HUB — incidents, tickets, documents, automation executions, assets, alerts, workflows, runbooks, notifications. There is **no per-module collaboration system**: every module reuses this one layer.

How it works

Collaboration is addressed **polymorphically** by `(entity_type, entity_uuid)` :

- **Comments** (`collaboration_comments`) — author, body, and mentioned user UUIDs. Mentioned users automatically become **watchers** (deterministic).
- **Participants** (`collaboration_participants`) — `watcher` / `assignee` / `owner` roles on an entity, uniquely constrained so a role is never duplicated.

`CollaborationService` exposes `thread()`, `comment()`, `addParticipant()`, `removeParticipant()`. All operations are workspace-scoped, UUID-only, and audited via the shared platform audit logger.

Reuse in the UI

The reusable `CollaborationPanel` React component takes `{ workspaceUuid, entityType, entityUuid }` and renders the comment thread + participants. It is embedded wherever collaboration is needed (e.g. the Service Desk ticket view). Embedding it in a new surface is a one-line component usage.

API (UUID-only, `workspace` required)

Base: `/api/collaboration` (Sanctum + `throttle:ai-workspace`). **Platform-wide**: available to any workspace member (`accessAi`) — there is no per-module entitlement, because collaboration is a shared capability every module reuses.

Method	Endpoint	Purpose
GET	<code>/thread?entity_type=&entity_uuid=</code>	Comments + participants for an entity
POST	<code>/comments</code>	Add a comment (with optional <code>mentions[]</code>)
POST	<code>/participants</code>	Add a watcher/assignee/owner
DELETE	<code>/participants</code>	Remove a participant role

Allowed `entity_type` values: `incident`, `ticket`, `document`, `execution`, `asset`, `alert`, `workflow`, `runbook`, `notification`. Roles: `watcher`, `assignee`, `owner`.

Guarantees

One shared layer (no duplication) · polymorphic over any entity · workspace isolation · RBAC · UUID-only · audited · EN/AR.